Tri3GN-UGC Algorithm: Complete Implementation Details

Algorithm 1: Tri3GN-UGC Coarsening and Supernode Allocation

Input:

- **`A`** — adjacency matrix (n × n)
- **`X`** — node feature matrix (n × f)
- **`y`** — node labels (used for homophily computation)
- **`cr`** — coarsening ratio = 0.5
- **`R`** — hops = 3
- **`d`** — branch dimension = 128
- **`seed`** — 7

Output:

- **`A_c`** — coarse adjacency matrix (m × m)
- **`c`** — cluster assignment vector c ∈ {1, ..., m}^n

Fixed Configuration:

- Coarsening ratio: cr = 0.5
- Eigenvalues: top k = 100 largest Laplacian eigenvalues
- Branch dimension: 128
- Hops: R = 3
- Hop weights (far): β = [1/6, 2/6, 3/6]
- Sketch repeats: 2
- Large-graph threshold: 6000 nodes
- Allocation: nearest-neighbor disjoint matching
- Coarse adjacency: normalized assignment, A_c = P^T A P
- Spectral calibration: mean eigenvalue scaling
- Device: CUDA when available
- Seed: 7

Step-by-Step Implementation

Step 1 — Preprocess Adjacency

```
A = symmetrize(A)
A = remove_self_loops(A)
A = set_nonzero_to_one(A)      # binary adjacency
```

- Ensure A is symmetric: A ← (A + A^T) / 2, then binarize
- Remove diagonal entries
- Set all nonzero entries to 1 (unweighted)

Step 2 — Compute Edge Homophily

```
h = |{(u, v) ∈ E : y_u = y_v}| / |E|
```

- Iterate over all edges (u, v)
- Count edges where source and target share the same label
- Divide by total number of edges |E|
- Result: h ∈ [0, 1]

Step 3 — Compute Bernstein Branch Weights

Using degree-2 Bernstein basis parameterized by h:

```
α_X  = (1 - h)²
α_A  = 2h(1 - h)
α_AX = h²
```

Properties:

- α_X + α_A + α_AX = 1 (sum to one, no fixed prior)
- When h → 0 (heterophilic): α_X dominates (raw features)
- When h → 1 (homophilic): α_AX dominates (multi-hop propagated features)
- When h = 0.5 (mixed): α_A peaks (structural identity)

Step 4 — Build Three Row-Normalized Branches

Branch 1: Feature Branch Z_X

```
Z_X = row_norm(Π_X · row_norm(X))
```

- Π_**X** = Johnson-Lindenstrauss random projection matrix (f → 128 dimensions)
  - Draw Π_X ∈ R^{128 × f} with entries sampled from N(0, 1/128)
  - Use seed=7 for reproducibility
- First apply row normalization to X: each row divided by its L2 norm
- Then project: (n × f) · (f × 128) → (n × 128)
- Apply row normalization again to the result

Branch 2: Structure Branch Z_A

```
Z_A = row_norm(CountSketch(A))
```

- Apply CountSketch to the adjacency matrix A
  - CountSketch repeats = 2
  - Maps each row of A (sparse, length n) to a dense sketch vector of length 128
  - Each repeat hashes column indices to sketch bins and accumulates ±1 signed contributions
  - Average over repeats
- Apply row normalization to the sketch output
- This captures structural identity (neighborhood fingerprint) without forming a dense n × n matrix

Branch 3: Multi-hop Branch Z_AX

```
Ã = D^{-1/2} A D^{-1/2}          # normalized adjacency, no self-loops
```

```
H = Σ_{r=1}^{R} β_r · Ã^r · X   # weighted multi-hop aggregation

Z_AX = row_norm(Π_AX · row_norm(H))
```

- Compute normalized adjacency Ã:
  - D = degree matrix (diagonal), D_{ii} = Σ_j A_{ij}
  - Ã = D^{-1/2} A D^{-1/2}
- Compute far hop weights for R=3:

  `β_r = r / Σ_{j=1}^{R} j  →  β = [1/6, 2/6, 3/6]`

  - Hop 1 weight: 1/6
  - Hop 2 weight: 2/6
  - Hop 3 weight: 3/6 (farther hops weighted more)
- Compute H iteratively to avoid recomputing powers:

  `H = β_1·(Ã·X) + β_2·(Ã²·X) + β_3·(Ã³·X)`

Efficiently: accumulate Ã^r · X by repeated sparse-dense multiply

- Apply row normalization to H
- Project using $\Pi_{AX} \in \mathbb{R}^{128 \times f}$ (separate random projection, same construction as Π_X)
- Apply final row normalization

Step 5 — Fuse Branches

```
Z = row_norm(α_X · Z_X + α_A · Z_A + α_AX · Z_AX)
```

- Weighted sum of the three branches using Bernstein weights
- Apply a final row normalization to the fused representation
- Result $Z \in \mathbb{R}^{n \times 128}$

Step 6 — Set Target Supernodes

```
m = round(n · (1 - cr))     # target number of supernodes
q = n - m                    # number of disjoint pair merges required
```

- With cr = 0.5: m = round(n/2), q = n - m ≈ n/2
- q is the exact number of node pairs that must be merged

Step 7 — Candidate Generation (Small Graphs: n ≤ 6000)

```
D_ij = ||Z_i - Z_j||_2      # pairwise Euclidean distance in Z
```

- Compute full n × n pairwise Euclidean distance matrix in the fused space Z
- For each node i, keep its top-8 nearest candidate neighbors (smallest distances)
- Collect all candidate pairs (i, j) with i < j
- Sort all candidate pairs by distance (ascending)

Step 8 — Candidate Generation (Large Graphs: n > 6000)

```
v ~ Uniform(S^{d-1})         # random unit projection vector
score_i = Z_i · v            # projected score for node i
```

- Draw 12 random projection vectors
- For each projection:
  - Sort all n nodes by their projected score
  - Add pairs (sorted[k], sorted[k+1]) and (sorted[k], sorted[k+2]) for all k

(offsets 1 and 2 in sorted order)

  - Score each candidate pair by |score_i - score_j| (projected-score difference)
  - For each unordered pair (i,j), keep the best (smallest) score seen across all projections
- Final candidate list: all unique pairs, sorted by best score

---

Step 9 — Greedy Disjoint Matching

```
assigned = {False} × n
supernodes = []

for (i, j) in candidates (sorted by score, smallest first):
    if not assigned[i] and not assigned[j]:
        supernodes.append({i, j})
        assigned[i] = True
        assigned[j] = True
    if len(supernodes) == q:
        break
```

- Scan candidate pairs from smallest score to largest
- Accept pair (i, j) only if neither i nor j has already been assigned
- Each accepted pair becomes one supernode (merged pair)
- Stop after exactly q pairs are accepted

Step 10 — Fallback if Fewer Than q Pairs Accepted

If the greedy scan produces fewer than q accepted pairs:

Small graphs (n ≤ 6000):

```
remaining = unassigned nodes
D_remaining = exact pairwise distance matrix over remaining nodes
pair remaining nodes greedily by nearest neighbor
```

Large graphs (n > 6000):

```
draw one fresh random projection vector
sort remaining unassigned nodes by projected score
pair consecutive nodes in sorted order
also pair adjacent nodes (connected by an edge in A)
```

Continue until q total pairs are reached or no more pairings are possible.

Step 11 — Assign Singleton Supernodes

```
for each node i not yet assigned:
    supernodes.append({i})    # singleton supernode
```

- Every unmatched node becomes its own supernode
- Relabel all supernode IDs to be consecutive: 0, 1, 2, ..., m-1

Step 12 — Form Assignment Matrix and Normalized Assignment

```
C ∈ {0,1}^{n×m}    where C_{ij} = 1 if node i belongs to supernode j

s_j = Σ_i C_{ij}   # size of supernode j (1 for singletons, 2 for pairs)

P = C · diag(s_1^{-1/2}, ..., s_m^{-1/2})
```

- C is the binary cluster assignment matrix (sparse)
- s_j is the number of nodes in supernode j
- P is the normalized assignment matrix:
  - Each column of C is scaled by 1/sqrt(supernode_size)
  - Singletons (s_j=1): column scaled by 1/1 = 1
  - Pairs (s_j=2): column scaled by 1/√2

Step 13 — Compute Coarse Adjacency

```
A_c = P^T A P
A_c = remove_diagonal(A_c)     # remove self-loops
A_c = symmetrize(A_c)          # ensure symmetry
```

- Matrix multiply: (m × n) · (n × n) · (n × m) → (m × m)
- Remove diagonal self-loops from A_c
- Symmetrize: A_c ← (A_c + A_c^T) / 2

Return: A_c (coarse adjacency), c (cluster assignment vector)

Spectral Metric: Relative Eigen Error (REE)

```
L   = D - A         # original graph Laplacian
L_c = D_c - A_c     # coarse graph Laplacian

λ_1(L) ≥ λ_2(L) ≥ ... ≥ λ_k(L)     # top-k=100 eigenvalues of L
λ_1(L_c) ≥ ... ≥ λ_k(L_c)           # top-k=100 eigenvalues of L_c

γ = (1/k · Σ_{i=1}^{k} λ_i(L)) / (1/k · Σ_{i=1}^{k} λ_i(L_c))   # spectral calibration scale

REE = 1/k · Σ_{i=1}^{k} |λ_i(L) - γ·λ_i(L_c)| / |λ_i(L)|
```

- Compute top-k=100 largest eigenvalues of both Laplacians
- γ is the mean eigenvalue ratio (global scale correction)
- REE averages the relative pointwise eigenvalue error
- **Lower REE = better spectral preservation**

Pipeline Summary

```
pipeline = tri_bern2 / far / fuse / nn / norm / mean
```

Stage	Operation
`tri_bern2`	Three branches with degree-2 Bernstein weights from homophily
`far`	Far hop weighting β = [1/6, 2/6, 3/6] for multi-hop branch
`fuse`	Weighted sum fusion Z = row_norm(α_X·Z_X + α_A·Z_A + α_AX·Z_AX)
`nn`	Nearest-neighbor disjoint matching for supernode allocation
`norm`	Normalized assignment P = C·diag(s_j^{-1/2})
`mean`	Mean spectral calibration γ for REE computation

Run Commands

Full formula grid (Cora / Citeseer):

```
python tri3gn_ugc_research/tri3gn_ugc_experiments.py \
  --datasets cora citeseer \
  --preset formula \
  --out-dir tri3gn_ugc_research/results_tuning/formula \
  --device cuda \
  --k 100 \
  --branch-dim 128 \
  --exact \
  --threshold 6000
```

Compact formula grid (remaining datasets):

```
python tri3gn_ugc_research/tri3gn_ugc_experiments.py \
  --datasets texas cornell film wisconsin pubmed \
  --preset formula_core \
  --out-dir tri3gn_ugc_research/results_tuning/formula_core
```

Aggregation (after staged runs):

```
python aggregation_script.py
```

- Conda environment: **`video_env`**
- Staged run preserves exact candidate ordering and random seeds via saved CSVs